INDIAN SPACE RESEARCH ORGANIZATION

ISRO Telemetry, Tracking and Command Network

(ISTRAC)

Project Report on

"OPTIMIZATION OF AUTOCORRELATION FOR GNSS APPLICATION USING AI/ML TECHNIQUES"

Submitted By

Bindushree A

B.E in Computer Science and Engineering
(28/01/2026 – 27/05/2026)

Maharaja Institute of Technology Mysore

Under the Guidance of:

Dr. Dileep D
Scientist -Section Head, NTRS 1
ISTRAC/ISRO

Submitted to:

Mr. B. Sankar Madaswamy
Division Head, HRLD
ISTRAC/ISRO

ACKNOWLEDGMENTS

The project work was undertaken at the SCC, ISRO Telemetry Tracking and Command Network (ISTRAC), ISRO. Being part of this distinguished department provided me with invaluable exposure to the processes and technologies that ensure the success of India's space missions, offering a unique environment for learning and collaboration.

I am deeply thankful to **Dr. A. K. Anil Kumar**, Director, ISTRAC, for granting this project work opportunity and for his visionary leadership in fostering programs that encourage learning and innovation.

My appreciation extends to **Mr. B. Sankar Madaswamy,** Manager, HRD, ISTRAC, ISRO, for facilitating this incredible project work opportunity and ensuring the smooth administrative execution of the project.

I express my sincere gratitude to **Shri. Subramanya Ganesh T**, Deputy Director, NSA, Scientist/Engineer-G, for his valuable support and for providing the necessary resources to pursue this work.

I am highly grateful to **Smt. Roopa M V**, Group Director, NSA, Scientist/Engineer-G, for her encouragement and for fostering a supportive environment throughout the duration of the project.

I extend my heartfelt gratitude to my project guide, **Dr. Dileep Dharmappa**, Scientist/Engineer – Section Head, NTRS1, ISTRAC, ISRO, for his expert guidance, constant support, and the generous time he dedicated to mentoring me. I am sincerely thankful for the opportunity to learn under his supervision.

Lastly, I wish to thank the entire team at ISTRAC for their warm welcome, cooperation, and constant encouragement throughout this rewarding journey.

CERTIFICATE

This is to certify that the project titled – **"Autocorrelation Optimization for GNSS Application using AI/ML Techniques"** has been successfully carried out by **Bindushree A** (4MH22CS018), a bonafide student of **Maharaja Institute of Technology Mysore**, enrolled in the Bachelor of Engineering in Computer Science and Engineering program for the academic period 2022–2026.

The project work was undertaken at the **Indian Space Research Organization – ISRO Telemetry, Tracking and Command Network (ISTRAC), Bengaluru**, during the period **28nd January 2026 to 27th May 2026**, in partial fulfilment of the requirements of her academic program. The project work was specifically carried out under the guidance of **Dr.Dileep Dharmappa, Scientist/Engineer Section Head, NTRS1 ISRO–ISTRAC**.

Dr.Dileep Dharmappa,
Scientist Section Head, NTRS1
ISRO–ISTRAC

DECLARATION

I, Bindushree A (4MH22CS018), a student of Computer Science and Engineering of Maharaja Institute of Technology Mysore, hereby declare that the project work titled - 'Autocorrelation Optimization for GNSS Application using AI/ML Techniques' has been successfully completed by me at the Indian Space Research Organization – ISRO Telemetry, Tracking and Command Network (ISTRAC), Bengaluru, under the guidance of Dr.Dileep Dharmappa, Scientist– Section Head, NTRS1 ISRO–ISTRAC.

This project work was carried out during the period 28nd January, 2026 to 27th May, 2026 during the duration of my Bachelor of Engineering program.

Date: 27th May, 2026
Place: Bangalore

Bindushree A
4MH22CS018

TABLE OF CONTENTS

ABSTRACT

Behind every rocket launch and satellite image lies a quiet network of people and systems working tirelessly to keep India's space dreams alive. The ISRO Telemetry, Tracking and Command Network (ISTRAC) is one such backbone, ensuring that spacecraft remain connected to Earth during their most critical moments. From Chandrayaan's lunar discoveries to the Mars Orbiter's historic success, ISTRAC has been the invisible lifeline that turned ambition into achievement. Alongside it, NavIC — India's own navigation constellation — touches everyday lives, guiding fishermen safely home, helping farmers with precision agriculture, and strengthening national security by reducing reliance on foreign systems.

These achievements are not just about rockets and satellites; they are also about the mathematics and engineering that make modern communication possible. Concepts like **Galois Fields, Linear Feedback Shift Registers (LFSRs), primitive polynomials, and correlation techniques** may sound abstract, but they power technologies we use daily — from secure online transactions to error-free data transmission and GPS-like navigation. Together, they form the hidden language of reliability and trust in digital systems.

This project explores the synergy between India's space infrastructure and the mathematical foundations that support it. It highlights how human effort, scientific rigor, and teamwork transform complex theories into practical tools that shape both national progress and everyday life. In essence, it is a story of how India blends **innovation, resilience, and vision** to reach for the stars while staying firmly connected to the people on the ground.

INTRODUCTION

ISRO's ISTRAC is often called the "silent backbone" of India's space missions. While rockets, astronauts, and satellites usually receive public attention, organizations like ISTRAC work quietly behind the scenes to make every mission possible. Without its support, India's satellites and space probes would not be able to communicate with Earth, making it impossible to control or monitor them once they leave the ground.

ISTRAC stands for the **ISRO Telemetry, Tracking and Command Network**. It is a specialized division of the Indian Space Research Organisation, headquartered in Bengaluru. Its main responsibility is to stay connected with spacecraft during every stage of a mission. Whether it is a satellite orbiting Earth, a spacecraft heading toward Mars, or a lunar lander touching down on the Moon, ISTRAC acts like a communication bridge between scientists on Earth and machines in space.

To understand ISTRAC better, imagine a spacecraft as a traveller on a very long journey. Just as travellers need maps, communication, and guidance, spacecraft also need constant support. ISTRAC provides this support through telemetry, tracking, and command systems. Telemetry means receiving data from spacecraft, such as temperature, speed, fuel levels, and system health. Tracking involves observing the exact location and movement of the spacecraft. Command means sending instructions from Earth, such as changing direction, adjusting orbit, or activating instruments. These three activities together ensure that missions continue smoothly and safely.

The importance of ISTRAC grew as India's space ambitions expanded. In the early days of India's space program, only a few satellites were launched, and operations were relatively simple. But as ISRO began launching more advanced missions, the need for a dedicated communication and tracking network became clear. Over time, ISTRAC developed into a sophisticated system with ground stations and mission control facilities spread across India and other parts of the world.

One of the most exciting moments for ISTRAC came during Chandrayaan-1, India's first mission to the Moon. Launched in 2008, the spacecraft travelled hundreds of thousands of kilometers away from Earth. During this journey, ISTRAC engineers constantly monitored its condition and ensured communication remained stable. The mission became historic when it

helped confirm the presence of water molecules on the Moon's surface. Although scientists and astronauts often receive credit for such discoveries, teams at ISTRAC were equally important because they kept the spacecraft connected and functioning.

Another unforgettable achievement was the Mars Orbiter Mission in 2013. Reaching Mars is considered one of the most difficult challenges in space exploration because of the enormous distance involved. Signals traveling between Earth and Mars can take several minutes to arrive, and even small errors can lead to mission failure. ISTRAC handled the delicate task of tracking the spacecraft throughout its long journey. When India successfully entered Mars orbit on its first attempt, it was a proud moment not only for ISRO but also for every engineer working behind the scenes at ISTRAC.

More recently, ISTRAC played a major role in Chandrayaan-3, which achieved a successful soft landing near the Moon's south pole in 2023. During the tense landing sequence, scientists at mission control closely monitored incoming data from the spacecraft. Every signal received by ISTRAC brought relief and excitement. When the Vikram lander touched down safely, celebrations erupted across the country. For many people, it was a reminder that space missions are not only about machines and technology but also about teamwork, dedication, and human effort.

ISTRAC's work does not end after a mission is launched. It continues to support satellites throughout their operational life. India uses satellites for weather forecasting, communication, disaster management, television broadcasting, navigation, and environmental monitoring. These satellites help farmers predict rainfall, assist fishermen in navigation, improve internet and communication systems, and support rescue operations during natural disasters. ISTRAC ensures these satellites remain healthy and operational by continuously monitoring them and sending necessary commands when required.

The organization also operates powerful antennas and communication systems capable of receiving signals from spacecraft located millions of kilometers away. Some of these signals are extremely weak by the time they reach Earth. Engineers at ISTRAC must therefore rely on advanced technology and precision to interpret the data correctly. This requires not only scientific expertise but also patience and concentration.

One interesting aspect of ISTRAC is the human side of its operations. During major missions, scientists and engineers often work for long hours without rest. The pressure can be enormous because even a small mistake may affect an entire mission. Yet the teams continue working

with determination and calmness. Images from mission control rooms often show scientists anxiously watching screens, waiting for confirmation signals from spacecraft. These moments reveal the emotional side of space exploration — the tension, hope, relief, and joy experienced by the people behind the missions.

ISTRAC also represents India's growing confidence in space technology. Over the years, it has evolved from supporting basic satellite launches to handling complex interplanetary missions. Its achievements have earned international respect, and it frequently collaborates with global space agencies for tracking and mission support. Such partnerships highlight India's increasing importance in the international space community.

In the coming years, ISTRAC's responsibilities are expected to grow even further. India's planned Gaganyaan mission, which aims to send Indian astronauts into space, will require highly reliable communication and tracking systems. Human spaceflight demands even greater precision because astronauts' lives depend on flawless coordination. ISTRAC will therefore play a critical role in ensuring the safety and success of these future missions.

In conclusion, ISTRAC is much more than a technical network. It is the heartbeat of India's space missions, quietly supporting every satellite and spacecraft launched by ISRO. While rockets may capture public imagination, organizations like ISTRAC remind us that successful space exploration depends on teamwork, discipline, and constant communication. Its contributions have helped India achieve milestones that once seemed impossible, from reaching the Moon to orbiting Mars. As India continues its journey into deeper space, ISTRAC will remain one of the strongest pillars supporting the nation's dreams among the stars.

NavIC

ISRO's NavIC is one of India's most meaningful achievements in space technology because it connects directly with everyday life. Most people use navigation systems almost daily without thinking much about them. Whether someone is booking a cab, searching for directions, tracking a food delivery, or checking the fastest route home, navigation technology quietly works in the background. NavIC was created to ensure that India could provide these services independently, accurately, and securely through its own satellite network.

NavIC stands for **Navigation with Indian Constellation**. It is India's own regional navigation system developed by the Indian Space Research Organisation. In simple terms, NavIC works like a guide in the sky. A group of satellites orbiting Earth constantly sends signals to receivers on the ground, helping users identify their exact location and direction. It is often compared to the American GPS because both systems perform similar functions, but NavIC was specially designed to serve India and nearby regions with high accuracy.

The idea behind NavIC came from an important realization. For many years, India depended mostly on foreign navigation systems such as GPS. While these systems were useful, depending completely on another country for such an essential service was risky. During emergencies, military conflicts, or natural disasters, access to navigation information becomes extremely important. India understood that it needed its own reliable system that would always remain under national control. This thought eventually led to the development of NavIC.

Building a navigation system is not easy. It requires years of scientific research, engineering, and teamwork. Scientists had to design satellites capable of sending precise timing signals from space. Ground stations had to be established to monitor the satellites continuously. Engineers also needed to create highly accurate atomic clocks because even a tiny timing error could affect navigation accuracy. Despite these challenges, Indian scientists successfully built NavIC step by step, proving the country's growing technological strength.

The first NavIC satellite was launched in 2013, and additional satellites followed over the next few years. Together, they formed a constellation that could provide uninterrupted navigation services across India and surrounding regions. Unlike GPS, which covers the entire world, NavIC mainly focuses on India and areas extending about 1,500 kilometers beyond its borders.

This regional focus allows it to provide very accurate location information within its service area.

One of the reasons NavIC is special is because it is not just about technology; it is about people. Its benefits can be seen in many parts of daily life. Imagine a fisherman going far into the sea early in the morning. In the past, poor weather or navigation mistakes could put lives at risk. With NavIC-enabled devices, fishermen can track their routes more safely and avoid accidentally crossing international maritime borders. During cyclones or storms, warning messages can also be sent quickly, giving fishermen enough time to return safely. In this way, NavIC becomes more than a machine-based system — it becomes a tool that protects lives.

Drivers and travelers also benefit from NavIC. Anyone who has experienced getting lost in traffic understands how useful accurate navigation can be. NavIC helps provide reliable directions, reduces travel confusion, and improves transportation systems. Emergency services can use accurate location data to reach accident sites faster, which can sometimes save precious minutes during medical emergencies.

In farming communities, NavIC has opened new possibilities. Agriculture today increasingly depends on technology. Farmers can use satellite-based navigation for precision farming, where water, fertilizers, and pesticides are used more efficiently. This not only improves crop production but also reduces waste and environmental damage. For a country like India, where agriculture supports millions of livelihoods, such technological support can make a significant difference.

NavIC also becomes extremely important during natural disasters. India frequently experiences floods, cyclones, earthquakes, and landslides. During such situations, rescue teams need accurate maps and location data to reach affected areas quickly. Navigation systems help coordinate relief operations, transport supplies, and identify safe routes. When communication systems are under pressure during emergencies, reliable satellite-based services become even more valuable.

Beyond civilian use, NavIC has an important role in national security. Modern defense systems rely heavily on navigation and timing technology. Aircraft, ships, drones, and missiles all need precise positioning data. If a country depends entirely on foreign systems during conflict situations, there is always uncertainty about signal availability. By developing NavIC, India ensured that its defense forces would always have access to independent navigation support when needed.

Another inspiring aspect of NavIC is what it says about India's scientific journey. Developing such a system placed India among a small group of countries with independent satellite navigation capabilities. It showed that Indian scientists and engineers could handle highly complex technologies that only a few nations in the world possess. The success of NavIC became a source of national pride because it represented self-reliance and innovation.

Today, NavIC is gradually becoming more visible in everyday technology. Several smartphones and electronic devices now support NavIC signals. This means ordinary users can directly benefit from India's own navigation system while using maps, travel apps, and location-based services. The government has also encouraged industries and transport systems to adopt NavIC technology more widely.

Of course, the journey has not been completely smooth. Some satellites experienced technical problems, especially with atomic clocks that are essential for maintaining timing precision. Replacing and maintaining satellites requires constant investment and effort. Since GPS has dominated global navigation for decades, convincing manufacturers and companies to adopt NavIC also takes time. However, Indian scientists continue improving the system with newer satellites and upgraded technology.

What makes NavIC truly meaningful is its quiet presence in everyday life. Most people using navigation services may not even realize which satellites are helping them. Yet behind every accurate location, safe fishing trip, faster emergency response, or guided journey, there is a network of scientists, engineers, and satellites working continuously. NavIC represents years of dedication by thousands of people who believed India could create world-class technology on its own.

NavIC also inspires future generations. Students interested in science and technology often see projects like NavIC as proof that Indian innovation can compete globally. It encourages young minds to dream bigger, whether in engineering, space science, artificial intelligence, or communication technology. In many ways, NavIC is not just about satellites in space; it is about building confidence on the ground.

Looking ahead, NavIC is expected to play an even larger role in India's digital future. As smart transportation systems, autonomous vehicles, drone delivery services, and advanced communication networks grow, accurate navigation will become even more important. NavIC has the potential to support these developments while reducing dependence on foreign systems.

In conclusion, NavIC is much more than a navigation project. It is a story of India's determination to become technologically independent and socially connected through space science. Developed by the Indian Space Research Organisation, NavIC supports daily life in ways many people may never notice — guiding travelers, helping farmers, protecting fishermen, supporting rescue teams, and strengthening national security. Its success reflects not only scientific achievement but also the human effort, vision, and persistence behind India's growing space journey.

GALOIS FIELD

A Galois Field, commonly called a finite field, is a mathematical system that contains a limited number of elements and allows operations such as addition, subtraction, multiplication, and division to be performed within that system. The concept is named after the French mathematician Évariste Galois, whose pioneering work in algebra laid the foundation for modern finite field theory. Today, Galois Fields are widely used in computer science, communication engineering, coding theory, and cryptography.

In ordinary mathematics, number systems such as real numbers contain infinitely many elements. In contrast, a Galois Field has only a finite set of elements. A finite field is represented as GF(q), where "GF" stands for Galois Field and q represents the total number of elements in the field. The value of q must be either a prime number or a power of a prime number.

The simplest and most commonly used finite field is GF(2), which contains only two elements: 0 and 1. Arithmetic operations in GF(2) follow modulo-2 rules. This means calculations wrap around after reaching 2. For example:

- $1 + 1 = 0$
- $1 + 0 = 1$
- $0 + 0 = 0$

These operations are very important in digital systems because computers work naturally with binary values. Logic circuits, digital communication systems, and storage devices all depend on binary arithmetic, making GF(2) especially useful.

More advanced finite fields are represented as $GF((2^n))$ or $GF((p^n))$, where p is a prime number and n is a positive integer. These fields are created using irreducible or primitive polynomials. For example, $GF(2^3)$ contains eight elements and is generated using polynomial arithmetic.

One major application of Galois Fields is error correction coding. During data transmission, signals may become corrupted due to noise or interference. Coding techniques based on finite fields help detect and correct these errors automatically. Technologies such as CDs, DVDs, QR codes, satellite communication systems, and deep-space probes rely heavily on Galois Field arithmetic for reliable data transmission.

Cryptography is another important application area. Modern encryption algorithms, including the Advanced Encryption Standard, use finite field operations to secure digital information. Finite fields provide mathematical structures that are efficient, predictable, and difficult to break without proper keys.

Galois Fields are also used in pseudorandom sequence generation, spread-spectrum communication, wireless systems, and digital signal processing. In communication engineering, sequences generated over finite fields help reduce interference and improve signal reliability.

One of the key mathematical properties of a finite field is that every nonzero element has a multiplicative inverse. This ensures division is always possible except by zero, making the structure algebraically complete.

Although Galois Fields may appear abstract at first, they are deeply connected to modern technology. Every time someone scans a QR code, stores files digitally, or uses secure online communication, finite field mathematics is involved in the background. Their combination of elegant mathematics and practical usefulness makes Galois Fields one of the most important concepts in modern digital engineering.

LINEAR FEEDBACK SHIFT REGISTER

A Linear Feedback Shift Register, commonly known as an LFSR, is a digital circuit used to generate sequences of binary numbers. It is widely used in communication systems, cryptography, digital electronics, and error detection systems because of its simplicity and efficiency. Despite its relatively basic structure, an LFSR can generate long and complex pseudorandom sequences that are extremely useful in modern technology.

An LFSR mainly consists of shift registers connected with feedback paths. A shift register is a sequence of flip-flops that store binary data. During each clock cycle, the bits shift from one position to the next. The “feedback” part comes from selected bits of the register being combined using XOR (exclusive OR) operations and then fed back into the input of the register.

For example, consider a 4-bit LFSR. Initially, the register contains a binary pattern called the seed value. At every clock pulse, the bits shift one position, and a new bit enters based on the

XOR of selected tap positions. This process continues repeatedly, generating a sequence of binary outputs.

One of the most important features of an LFSR is its ability to produce pseudorandom sequences. These sequences appear random even though they are generated deterministically. If the feedback taps are chosen correctly using primitive polynomials, the LFSR produces a maximum-length sequence, also called an m-sequence.

For an n-bit LFSR, the maximum sequence length is:

$2^{n} - 1$

This means a 5-bit LFSR can generate 31 different states before the sequence repeats.

LFSRs are widely used in communication engineering. In spread-spectrum systems such as CDMA, pseudorandom sequences generated by LFSRs help multiple users share the same communication channel without significant interference. These sequences also help improve security and resistance to noise.

In cryptography, LFSRs are used for stream cipher design. A stream cipher encrypts data by combining plaintext with a pseudorandom keystream. Although a single LFSR is not secure enough for advanced cryptographic applications, combining multiple LFSRs creates stronger encryption methods.

Another important application of LFSRs is error detection. Cyclic Redundancy Check (CRC) systems use structures similar to LFSRs to detect errors in transmitted data. Computer networks, storage devices, and digital communication protocols commonly use CRC methods.

LFSRs are also useful in hardware testing. Built-in self-test systems generate test patterns using LFSRs to verify whether digital circuits function correctly. Since LFSRs require very little hardware, they are cost-effective and fast.

The efficiency of an LFSR depends heavily on the feedback polynomial used. Primitive polynomials ensure that the generated sequence achieves maximum length and good statistical properties.

Despite their usefulness, LFSRs have limitations. Because they are deterministic, their outputs can eventually be predicted if the structure and initial state are known. Therefore, secure systems often combine multiple LFSRs with nonlinear techniques.

In conclusion, the Linear Feedback Shift Register is a simple yet powerful digital sequence generator. Its applications in communication systems, cryptography, error detection, and digital testing make it one of the most important building blocks in modern digital engineering.

PRIMITIVE POLYNOMIAL

A primitive polynomial is a special type of polynomial used in finite field mathematics, coding theory, cryptography, and digital communication systems. Primitive polynomials are essential because they help generate finite fields and produce maximum-length pseudorandom sequences used in many engineering applications.

To understand primitive polynomials, it is important first to understand polynomials over finite fields. In binary systems, coefficients are limited to values such as 0 and 1. For example:

$x^3 + x + 1$

is a polynomial defined over GF(2).

A primitive polynomial must satisfy two important conditions. First, it must be irreducible, meaning it cannot be factored into smaller polynomials within the same field. This property is similar to prime numbers in ordinary arithmetic. Second, the polynomial must generate all nonzero elements of the finite field through repeated operations. When both conditions are satisfied, the polynomial is called primitive.

Primitive polynomials are extremely important in constructing extension fields such as $GF(2^3)$, $GF(2^4)$, and higher-order fields. These finite fields are widely used in digital communication and error correction systems.

One of the most common applications of primitive polynomials is in Linear Feedback Shift Registers (LFSRs). The feedback taps of an LFSR are selected according to a primitive polynomial. If the polynomial is primitive, the LFSR generates a maximum-length sequence called an m-sequence.

For an n-bit LFSR, the maximum possible sequence length is:

2^{n-1}

These long sequences are valuable because they have good randomness and correlation properties. They are widely used in spread-spectrum communication systems, satellite communication, radar systems, and secure data transmission.

Primitive polynomials also play an important role in cryptography. Encryption systems often rely on finite field arithmetic, where primitive elements generated by primitive polynomials provide strong mathematical structures for secure operations. Stream ciphers commonly use these polynomials to generate pseudorandom keystreams.

Another major application is error correction coding. Codes such as BCH codes and Reed–Solomon codes are built using finite fields generated by primitive polynomials. These codes help detect and correct transmission errors in CDs, DVDs, QR codes, mobile communication systems, and deep-space missions.

An example of a primitive polynomial over GF(2) is:

$x^4 + x + 1$

This polynomial generates all nonzero elements of $GF(2^4)$. When used in a 4-bit LFSR, it produces a sequence of length 15 before repeating.

Not every irreducible polynomial is primitive. Some irreducible polynomials fail to generate maximum-length sequences. Therefore, mathematical testing is necessary to identify primitive polynomials correctly.

Primitive polynomials are also connected to cyclic structures in algebra. In finite fields, nonzero elements form cyclic multiplicative groups, and primitive polynomials help identify generators of these groups.

Although the mathematics behind primitive polynomials may seem abstract, their practical applications are enormous. They support reliable communication, secure encryption, and efficient sequence generation in modern digital systems.

In conclusion, primitive polynomials are powerful algebraic tools that form the foundation of finite field arithmetic and digital sequence generation. Their importance in communication systems, coding theory, and cryptography makes them essential in modern information technology and digital engineering.

AUTOCORRELATION AND CROSS-CORRELATION

Autocorrelation and cross-correlation are important mathematical techniques used in signal processing, communication engineering, statistics, radar systems, image processing, and digital communication. Both methods are used to measure similarity between signals, but they differ in the type of comparison they perform. These concepts help engineers analyze patterns, detect signals, remove noise, and improve communication reliability.

Autocorrelation measures the similarity of a signal with a delayed version of itself. In simple terms, it checks how closely a signal matches itself after being shifted in time. If a signal strongly resembles its shifted version at certain intervals, the autocorrelation value becomes high. This technique is especially useful for identifying repeating patterns or periodicity in signals.

Mathematically, the autocorrelation of a discrete signal is represented as:

R_{xx}(k)=sum x(n)x(n-k)

where:

- (x(n)) is the original signal,
- (k) is the time shift or lag,
- (R_{xx}(k)) is the autocorrelation function.

One important property of autocorrelation is that its maximum value occurs when the shift is zero because the signal perfectly matches itself. As the shift increases, similarity may decrease depending on the nature of the signal.

Autocorrelation is widely used in communication systems and pseudorandom sequence analysis. In spread-spectrum communication systems such as CDMA, ideal pseudorandom sequences have a sharp autocorrelation peak at zero shift and low values elsewhere. This property helps receivers detect the correct signal even in the presence of noise and interference.

Another important application is radar and sonar systems. Reflected signals are correlated with transmitted signals to determine the presence, distance, and speed of objects. In speech processing and music analysis, autocorrelation helps identify pitch and periodic sound patterns.

Cross-correlation, on the other hand, measures the similarity between two different signals. Instead of comparing a signal with itself, it compares one signal with another shifted version.

This helps determine whether two signals are related and identifies the amount of delay between them.

The mathematical expression for cross-correlation is:

R_{xy}(k)=\sum x(n)y(n-k)

where:

- (x(n)) and (y(n)) are two different signals,
- (k) represents the shift,
- (R_{xy}(k)) is the cross-correlation value.

If two signals are highly similar, the cross-correlation function produces a large value at the shift where they align best. If the signals are unrelated, the values remain low.

Cross-correlation is extremely useful in communication engineering. Receivers use cross-correlation to identify incoming signals and synchronize communication systems. In GPS and satellite communication, receivers correlate received signals with known reference sequences to determine timing and position accurately.

Image processing also uses cross-correlation for pattern recognition and template matching. For example, facial recognition systems compare image patterns using correlation techniques. In medical signal analysis, cross-correlation helps compare biological signals such as ECG or EEG data.

One key difference between autocorrelation and cross-correlation is that autocorrelation always compares a signal with itself, while cross-correlation compares two separate signals. Autocorrelation mainly identifies periodicity and self-similarity, whereas cross-correlation identifies relationships between different signals.

In conclusion, autocorrelation and cross-correlation are essential tools in signal analysis and communication engineering. They help detect patterns, measure signal similarity, synchronize systems, and improve data reliability. From wireless communication and radar systems to image processing and biomedical applications, these techniques play a major role in modern technology and digital systems.

OPTIMIZATION OF PERIODIC AUTOCORRELATION

Optimization of periodic autocorrelation is a key concept in digital communications, coding theory, radar systems, and spread-spectrum signal design [1]. It focuses on designing or selecting signals whose periodic autocorrelation properties are "ideal" or near-ideal for specific engineering applications.

Periodic autocorrelation measures how similar a sequence is to a shifted version of itself when the sequence is treated as cyclic. Unlike aperiodic autocorrelation, where shifts go beyond signal boundaries and values are truncated, periodic autocorrelation assumes the sequence repeats infinitely in a circular manner [2]. This makes it especially important in systems where signals are naturally periodic, such as in code division multiple access systems, synchronization sequences, and pseudorandom sequence generators.

The main goal of optimization is to design sequences where the periodic autocorrelation has a sharp peak at zero shift and very low values for all other shifts [3]. This property is crucial because it allows receivers to detect the correct alignment of a signal in the presence of noise, interference, or overlapping transmissions. Ideally, the autocorrelation function should behave like a delta function in discrete form.

Optimization involves minimizing the magnitude of sidelobes (values of autocorrelation at non-zero shifts) [4]. These sidelobes cause interference and reduce system performance in practical applications. Therefore, sequence design aims to reduce them as much as possible.

One of the most important approaches to optimization is the use of algebraic structures such as Linear Feedback Shift Register sequences generated using primitive polynomials [5]. These sequences, often called m-sequences, exhibit nearly ideal periodic autocorrelation properties with a single strong peak and uniform sidelobe distribution. However, even these are not perfectly zero outside the main peak, so further optimization is often required.

Another approach involves using mathematical constructions such as difference sets, Golomb sequences, and cyclic codes [1][2]. These methods aim to balance the distribution of values within a sequence so that correlation properties are improved. In advanced applications, optimization is achieved using computational techniques such as genetic algorithms [6], simulated annealing [7], and convex optimization, which search for sequences with minimal autocorrelation sidelobes.

In communication systems, optimized periodic autocorrelation is essential for synchronization. When a receiver tries to align with a transmitted signal, a sharp autocorrelation peak ensures accurate timing detection. In radar and sonar systems, it helps improve range resolution by clearly distinguishing reflected signals from different objects.

In cryptography and secure communication, optimized sequences help reduce predictability while maintaining desirable randomness properties [4]. This ensures both efficiency and security in data transmission.

In conclusion, optimization of periodic autocorrelation is about designing sequences with maximum distinguishability and minimal interference [3]. By using algebraic methods, sequence theory, and computational optimization, engineers create signals that are robust, efficient, and highly reliable for modern digital communication and signal processing systems.

In addition to classical optimization methods, recent research has increasingly incorporated AI/ML-based and metaheuristic techniques to improve periodic autocorrelation performance beyond what purely algebraic constructions can achieve [8]. These methods treat sequence design as a high-dimensional optimization problem where the objective is to minimize sidelobe levels and improve correlation properties under given constraints.

One of the approaches used is the Brest–Bošković strategy combined with metaheuristic search [9], where candidate solutions are iteratively refined using adaptive parameter control and stochastic exploration. This helps in efficiently navigating large search spaces that are otherwise computationally expensive using deterministic methods.

Another widely used technique is the Genetic Algorithm (GA) with local search enhancement [6][10], where an initial population of sequences is evolved through selection, crossover, and mutation. After each evolutionary cycle, local search is applied to fine-tune promising candidates, improving convergence toward sequences with lower periodic autocorrelation sidelobes.

Hybrid frameworks such as PeCAN (Population-based Evolutionary Crossover and Neighborhood search) integrated with Large Neighborhood Search (LNS) have also been employed [8][10]. These methods enhance exploration by generating diverse candidate solutions while simultaneously exploiting local structure to refine high-quality sequences. This balance helps avoid premature convergence and improves overall optimization quality.

Additionally, Hybrid Simulated Annealing techniques are used [7][9], where probabilistic acceptance of worse solutions allows the algorithm to escape local minima. Over time, the acceptance probability is reduced, enabling convergence toward highly optimized sequence structures with improved autocorrelation behavior.

These AI/ML-driven optimization methods are often initialized using well-structured algebraic sequences such as Legendre, Weil, and Sidelnikov sequences [1][2]. These initial seeds provide strong mathematical properties, while the optimization algorithms iteratively adjust and refine them to further suppress sidelobes and enhance correlation performance.

LEARNING OUTCOMES

This study provides a clear understanding of how India's space communication and navigation systems operate and how different mathematical and engineering concepts support them. It explains the role of ISRO Telemetry, Tracking and Command Network in maintaining continuous communication with satellites and spacecraft through telemetry, tracking, and command operations, ensuring smooth mission control from launch to deep-space operations.

It also builds understanding of NavIC, including how satellite-based navigation works and how it is used in real-life applications such as transportation, agriculture, disaster management, and defence systems. The importance of India's independent navigation capability and its practical benefits in daily life is also highlighted.

The report further develops knowledge of mathematical foundations such as Galois Field and their role in digital communication, coding theory, and cryptography. It also explains how Linear Feedback Shift Register and primitive polynomials are used to generate pseudorandom sequences for secure and efficient communication systems.

In addition, it strengthens understanding of autocorrelation and cross-correlation techniques used in signal analysis for synchronization, pattern detection, and noise reduction. The role of advanced optimization methods, including AI/ML-based metaheuristic techniques, is also covered to show how modern computational approaches improve sequence design and system performance.

CONCLUSION

In conclusion, this study shows how space systems, navigation technology, and digital communication are deeply connected through both engineering and mathematics. The ISRO Telemetry, Tracking and Command Network acts as the backbone of India's space missions by ensuring continuous monitoring and control of satellites and spacecraft, making complex missions possible and reliable.

The development of NavIC reflects India's progress toward self-reliance in navigation technology. It provides accurate positioning services that support everyday applications as well as critical sectors like disaster management, transportation, and national security.

At the core of these technologies are mathematical concepts such as Galois Field, Linear Feedback Shift Register, primitive polynomials, and correlation techniques. These concepts make it possible to design reliable communication systems, detect errors, and generate secure sequences.

Modern improvements using AI/ML-based optimization methods further enhance these systems by improving sequence design and reducing interference. Overall, the combination of space technology, mathematical theory, and intelligent optimization techniques forms the foundation of modern communication, navigation, and signal processing systems.

APPENDIX

BREST BOSKOVIC WITH METAHEURESTIC

A. LEGENDRE SEQUENCE

```
import numpy as np
import time
import os

def get_unified_seed(n):
    p = n
    while not all(p % i != 0 for i in range(2, int(p**0.5) + 1)): p += 1
    s = np.ones(p)
    for i in range(1, p):
        if pow(i, (p - 1) // 2, p) != 1: s[i] = -1
    seed = np.roll(s[:n], n // 4)
    return seed.astype(float), p

def write_wrapped_bits(f, seq):
    bin_str = "".join(['1' if x > 0 else '0' for x in seq])
    for i in range(0, len(bin_str), 100):
        f.write(bin_str[i:i+100] + "\n")

def run_brest_boskovic():
    n = int(input("Enter N: ") or "1024")
    initial_seq, p = get_unified_seed(n)

    f_init = np.fft.fft(initial_seq)
    acf = np.round(np.real(np.fft.ifft(f_init * np.conj(f_init)))).astype(float)
    init_psl = int(np.max(np.abs(acf[1:n//2+1])))

    seq = initial_seq.copy()
    best_psl = init_psl
```

```
    best_seq = seq.copy()
    gamma = 4.0

    print(f"Base Prime: {p} | Initial PSL: {init_psl}")

    for step in range(1, 300001):
        idx = np.random.randint(0, n)
        diff = -2 * seq[idx]
        indices_minus = (idx - np.arange(n)) % n
        indices_plus = (idx + np.arange(n)) % n
        delta = diff * (seq[indices_minus] + seq[indices_plus])

        sidelobes = slice(1, n // 2 + 1)
        if np.sum(np.abs(acf[sidelobes] + delta[sidelobes])**gamma) < np.sum(np.abs(acf[sidelobes])**gamma):
            seq[idx] *= -1
            acf += delta
            curr_psl = int(np.max(np.abs(acf[sidelobes])))
            if curr_psl < best_psl:
                best_psl = curr_psl
                best_seq = seq.copy()
                gamma = min(18, gamma + 0.1)
                print(f"Step {step} | PSL: {best_psl}")

    filename = f"BrestBoskovic_N{n}_PSL{best_psl}.txt"
    with open(filename, "w") as f:
        f.write(f"BREST-BOSKOVIC REPORT\nN: {n} | Base Prime: {p}\n")
        f.write(f"Initial PSL: {init_psl} | Final PSL: {best_psl}\n\n")
        f.write("--- INITIAL SEQUENCE ---\n")
        write_wrapped_bits(f, initial_seq)
        f.write("\n--- OPTIMIZED SEQUENCE ---\n")
        write_wrapped_bits(f, best_seq)
        f.flush(); os.fsync(f.fileno())
    print(f"Saved: {filename}")
```

```
if __name__ == "__main__": run_brest_boskovic()
```

B. WEIL SEQUENCE

```
import numpy as np
import os

def get_best_weil_seed(n):
    def is_prime(num):
        if num < 2: return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0: return False
        return True
    def get_primitive_roots(p):
        roots = []; phi = p - 1; temp = phi; factors = []; d = 2
        while d * d <= temp:
            if temp % d == 0:
                factors.append(d)
                while temp % d == 0: temp //= d
            d += 1
        if temp > 1: factors.append(temp)
        for res in range(2, p):
            ok = True
            for f in factors:
                if pow(res, phi // f, p) == 1:
                    ok = False; break
            if ok: roots.append(res)
        return roots
    p = n
    while not is_prime(p): p += 1
    roots = get_primitive_roots(p)
    best_psl = float('inf'); best_seed = None; best_g = None
    for g in roots:
        s = np.ones(p)
```

```
        for i in range(p):
            val = (pow(i, g, p) + 1) % p
            s[i] = 1 if (val == 0 or pow(val, (p - 1) // 2, p) == 1) else -1
        cand = np.roll(s[:n], n // 4)
        curr_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(cand)*np.conj(np.fft.fft(cand))
)))[1:n//2+1])))
        if curr_psl < best_psl:
            best_psl, best_seed, best_g = curr_psl, cand.astype(float), g
    return best_seed, p, best_g, n // 4

def run_brest_boskovic():
    n = int(input("Enter N: "))
    seed, p, g, shift = get_best_weil_seed(n)
    init_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(seed)*np.conj(np.fft.fft(seed))
)))[1:n//2+1])))

    seq = seed.copy(); best_seq = seed.copy(); best_psl = init_psl; gamma = 4.0
    f_vec = np.fft.fft(seq); acf = np.round(np.real(np.fft.ifft(f_vec *
np.conj(f_vec)))).astype(float)

    for step in range(1, 100001):
        idx = np.random.randint(0, n)
        diff = -2 * seq[idx]
        delta = diff * (seq[(idx - np.arange(n)) % n] + seq[(idx + np.arange(n)) % n])
        sl = slice(1, n // 2 + 1)
        if np.sum(np.abs(acf[sl] + delta[sl])**gamma) <
np.sum(np.abs(acf[sl])**gamma):
            seq[idx] *= -1; acf += delta
            curr_psl = int(np.max(np.abs(acf[sl])))
            if curr_psl < best_psl:
                best_psl, best_seq = curr_psl, seq.copy()
                gamma = min(18, gamma + 0.1)
```

```
                print(f"Step {step} | PSL: {best_psl}")

        with open(f"Brest_Weil_N{n}.txt", "w") as f:
            f.write(f"BREST-BOSKOVIC | N={n} | Prime={p} | Root={g} | Shift={shift}\n")
            f.write(f"Initial PSL: {init_psl} | Final PSL: {best_psl}\n\n")
            f.write("--- INITIAL WEIL ---\n")
            s_str = "".join(['1' if x > 0 else '0' for x in seed])
            for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
            f.write("\n--- OPTIMIZED ---\n")
            b_str = "".join(['1' if x > 0 else '0' for x in best_seq])
            for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
        print(f"Saved: Brest_Weil_N{n}.txt")

    if __name__ == "__main__": run_brest_boskovic()
```

C. SIDELNIKOV SEQUENCE

```
    import numpy as np
    import os

    def get_best_sidelnikov_seed(n):
        def is_prime(num):
            if num < 2: return False
            for i in range(2, int(num**0.5) + 1):
                if num % i == 0: return False
            return True
        def get_primitive_roots(p):
            roots = []; phi = p - 1; temp = phi; factors = []; d = 2
            while d * d <= temp:
                if temp % d == 0:
                    factors.append(d)
                    while temp % d == 0: temp //= d
                d += 1
            if temp > 1: factors.append(temp)
            for res in range(2, p):
```

```
            ok = True
            for f in factors:
                if pow(res, phi // f, p) == 1:
                    ok = False; break
            if ok: roots.append(res)
        return roots

    p = n
    while not is_prime(p): p += 1
    all_roots = get_primitive_roots(p)
    roots_to_scan = all_roots[:25]

    best_psl = float('inf'); best_seed = None; best_alpha = None

    for alpha in roots_to_scan:
        s = np.ones(p)
        log_table = {pow(alpha, i, p): i for i in range(p-1)}
        for i in range(p-1):
            val = (pow(alpha, i, p) + 1) % p
            if val == 0: s[i] = -1
            else:
                exponent = log_table[val]
                s[i] = 1 if (exponent % 2 == 0) else -1

        cand = np.roll(s[:n], n // 4)
        f = np.fft.fft(cand); acf = np.round(np.real(np.fft.ifft(f*np.conj(f)))).astype(int)
        curr_psl = int(np.max(np.abs(acf[1:n//2+1])))
        if curr_psl < best_psl:
            best_psl, best_seed, best_alpha = curr_psl, cand.astype(float), alpha

    return best_seed, p, best_alpha, n // 4

def run_brest_boskovic():
    n = int(input("Enter N: "))
```

```
    seed, p, alpha, shift = get_best_sidelnikov_seed(n)
    init_psl = int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(seed)*np.conj(np.fft.fft(seed))
))))[1:n//2+1])))

    seq = seed.copy(); best_seq = seed.copy(); best_psl = init_psl; gamma = 4.0
    f_vec = np.fft.fft(seq); acf = np.round(np.real(np.fft.ifft(f_vec * np.conj(f_vec)))).astype(float)

    for step in range(1, 80001):
        idx = np.random.randint(0, n)
        diff = -2 * seq[idx]
        delta = diff * (seq[(idx - np.arange(n)) % n] + seq[(idx + np.arange(n)) % n])
        sl = slice(1, n // 2 + 1)
        if np.sum(np.abs(acf[sl] + delta[sl])**gamma) < np.sum(np.abs(acf[sl])**gamma):
            seq[idx] *= -1; acf += delta
            curr_psl = int(np.max(np.abs(acf[sl])))
            if curr_psl < best_psl:
                best_psl, best_seq = curr_psl, seq.copy()
                gamma = min(18, gamma + 0.1)

    with open(f"Sidelnikov_Brest_N{n}.txt", "w") as f:
        f.write(f"METHOD: BREST-BOSKOVIC\nN: {n} | Prime: {p} | Selected Alpha: {alpha}\n")
        f.write(f"Initial PSL: {init_psl} | Final PSL: {best_psl}\n\n")
        f.write("--- INITIAL SEQUENCE ---\n")
        s_str = "".join(['1' if x > 0 else '0' for x in seed])
        for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
        f.write("\n--- FINAL SEQUENCE ---\n")
        b_str = "".join(['1' if x > 0 else '0' for x in best_seq])
        for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
    print(f"Brest-Boskovic Done. Alpha used: {alpha}")
```

```
if __name__ == "__main__": run_brest_boskovic()
```

PeCAN WITH LNS

A. LEGENDRE SEQUENCE

```
import numpy as np
import os
import time

def get_unified_seed(n):
    """Unified Seed: Search Up, Truncate, and Rotate n/4."""
    p = n
    while not all(p % i != 0 for i in range(2, int(p**0.5) + 1)):
        p += 1
    s = np.ones(p)
    for i in range(1, p):
        if pow(i, (p - 1) // 2, p) != 1:
            s[i] = -1
    # Truncate to N and apply the optimal Brest-Boskovic rotation
    seed = np.roll(s[:n], n // 4)
    return seed.astype(float), p

def get_metrics(s):
    """Fast Periodic ACF calculation."""
    n = len(s)
    S = np.fft.fft(s)
    acf = np.real(np.fft.ifft(S * np.conj(S)))
    rounded_acf = np.round(acf).astype(int)
    # Check sidelobes up to N/2
    psl = np.max(np.abs(rounded_acf[1:n//2+1]))
    return psl, rounded_acf

def write_wrapped_bits(f, seq):
```

```
    """Writes bits in 100-character blocks for file integrity."""
    bin_str = "".join(['1' if x == 1 else '0' for x in seq])
    for i in range(0, len(bin_str), 100):
        f.write(bin_str[i:i+100] + "\n")

def run_pecan_lns(n_target):
    # 1. Initialize with the Unified Seed (Matches other 3 methods)
    initial_seq, prime_p = get_unified_seed(n_target)
    init_psl, _ = get_metrics(initial_seq)

    print(f"\n[PHASE 1] Construction (Unified)")
    print(f"Target Length: {n_target} | Base Prime Core (p): {prime_p}")
    print(f"Initial PSL: {init_psl}")

    # 2. Optimization Loop (PeCAN + LNS)
    current_s = initial_seq.copy()
    best_s = initial_seq.copy()
    best_psl = init_psl

    stagnation = 0
    iteration = 0
    print(f"\n[PHASE 2] PeCAN+LNS Optimizing...")

    try:
        while stagnation < 1000:
            iteration += 1
            S = np.fft.fft(current_s)

            # --- PECAN CORE (Adaptive Spectral Shaping) ---
            noise = (stagnation / 1000) * 0.1
            target_mag = np.sqrt(n_target) + np.random.normal(0, noise, n_target)
            S_ideal = target_mag * np.exp(1j * np.angle(S))

            s_cont = np.real(np.fft.ifft(S_ideal))
```

```
        candidate_s = np.where(s_cont >= 0, 1.0, -1.0)

        curr_psl, _ = get_metrics(candidate_s)

        if curr_psl < best_psl:
            best_psl = curr_psl
            best_s = candidate_s.copy()
            stagnation = 0
            if iteration % 10 == 0:
                print(f"Iter {iteration}: New Best PSL = {best_psl}")
        else:
            stagnation += 1

        # --- LNS KICK (Local Neighborhood Search) ---
        if stagnation > 200 and stagnation % 50 == 0:
            # Flip bits with lowest confidence (nearest to zero)
            uncertainty = np.abs(s_cont)
            threshold = np.percentile(uncertainty, 1)
            candidate_s[uncertainty < threshold] *= -1

        current_s = candidate_s

    except KeyboardInterrupt:
        print("\nUser stopped search.")

    # 3. Final Reporting and Force-Write
    filename = f"PeCAN_LNS_N{n_target}_PSL{best_psl}.txt"
    with open(filename, "w") as f:
        f.write("PECAN + LNS OPTIMIZATION REPORT\n")
        f.write("="*50 + "\n")
        f.write(f"Target Length: {n_target}\n")
        f.write(f"Base Prime Used: {prime_p}\n")
        f.write(f"Initial PSL (Unified Seed): {init_psl}\n")
        f.write(f"Final Optimized PSL: {best_psl}\n")
```

```
            f.write("="*50 + "\n\n")

            f.write("--- INITIAL SEQUENCE (Unified Legendre) ---\n")
            write_wrapped_bits(f, initial_seq)

            f.write("\n--- FINAL OPTIMIZED SEQUENCE ---\n")
            write_wrapped_bits(f, best_s)

            f.flush()
            os.fsync(f.fileno())

        print(f"\nDONE! Final PSL: {best_psl}")
        print(f"Results saved to: {filename}")

    if __name__ == "__main__":
        N = int(input("Enter target sequence length: ") or "1024")
        run_pecan_lns(N)
```

B. WEIL SEQUENCE

```
    import numpy as np
    import os

    def get_best_weil_seed(n):
        # [Exactly same get_best_weil_seed logic as Method 1]
        def is_prime(num):
            if num < 2: return False
            for i in range(2, int(num**0.5) + 1):
                if num % i == 0: return False
            return True
        def get_primitive_roots(p):
            roots = []; phi = p - 1; temp = phi; factors = []; d = 2
            while d * d <= temp:
                if temp % d == 0:
                    factors.append(d)
```

```
            while temp % d == 0: temp //= d
        d += 1
    if temp > 1: factors.append(temp)
    for res in range(2, p):
        ok = True
        for f in factors:
            if pow(res, phi // f, p) == 1:
                ok = False; break
        if ok: roots.append(res)
    return roots
p = n
while not is_prime(p): p += 1
roots = get_primitive_roots(p)
best_psl = float('inf'); best_seed = None; best_g = None
for g in roots:
    s = np.ones(p)
    for i in range(p):
        val = (pow(i, g, p) + 1) % p
        s[i] = 1 if (val == 0 or pow(val, (p - 1) // 2, p) == 1) else -1
    cand = np.roll(s[:n], n // 4)
    curr_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(cand)*np.conj(np.fft.fft(cand))
)))[1:n//2+1])))
    if curr_psl < best_psl:
        best_psl, best_seed, best_g = curr_psl, cand.astype(float), g
return best_seed, p, best_g, n // 4


def run_pecan_lns():
    n = int(input("Enter N: "))
    seed, p, g, shift = get_best_weil_seed(n)
    init_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(seed)*np.conj(np.fft.fft(seed))
)))[1:n//2+1])))
```

```
    curr = seed.copy(); best_s = seed.copy(); best_psl = init_psl; stagnation = 0
    for i in range(1, 1001):
        S = np.fft.fft(curr)
        noise = (stagnation / 1000) * 0.1
        S_ideal = (np.sqrt(n) + np.random.normal(0, noise, n)) * np.exp(1j * np.angle(S))
        s_cont = np.real(np.fft.ifft(S_ideal))
        cand = np.where(s_cont >= 0, 1.0, -1.0)
        c_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(cand)*np.conj(np.fft.fft(cand))
)))[1:n//2+1])))

        if c_psl < best_psl:
            best_psl, best_s, stagnation = c_psl, cand.copy(), 0
            print(f"Iter {i} | PSL: {best_psl}")
        else: stagnation += 1

        if stagnation > 100 and stagnation % 50 == 0:
            cand[np.abs(s_cont) < np.percentile(np.abs(s_cont), 1)] *= -1
        curr = cand

    with open(f"PeCAN_Weil_N{n}.txt", "w") as f:
        f.write(f"PECAN + LNS | N={n} | Prime={p} | Root={g}\nInitial PSL: {init_psl}
| Final: {best_psl}\n")
        s_str = "".join(['1' if x > 0 else '0' for x in seed])
        for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
        f.write("\n--- OPTIMIZED ---\n")
        b_str = "".join(['1' if x > 0 else '0' for x in best_s])
        for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
    print(f"Saved: PeCAN_Weil_N{n}.txt")

if __name__ == "__main__": run_pecan_lns()
```

C. SIDELNIKOV SEQUENCE

```
import numpy as np
```

```
import os

def get_best_sidelnikov_seed(n):
    """
    Unified Sidelnikov Generator:
    Scans first 25 roots and picks the one with best starting PSL.
    """
    def is_prime(num):
        if num < 2: return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0: return False
        return True

    def get_primitive_roots(p):
        roots = []; phi = p - 1; temp = phi; factors = []; d = 2
        while d * d <= temp:
            if temp % d == 0:
                factors.append(d)
                while temp % d == 0: temp //= d
            d += 1
        if temp > 1: factors.append(temp)
        for res in range(2, p):
            ok = True
            for f in factors:
                if pow(res, phi // f, p) == 1:
                    ok = False; break
            if ok: roots.append(res)
        return roots

    p = n
    while not is_prime(p): p += 1
    all_roots = get_primitive_roots(p)
    roots_to_scan = all_roots[:25]
```

```
    best_psl = float('inf'); best_seed = None; best_alpha = None

    for alpha in roots_to_scan:
        s = np.ones(p)
        # Sidelnikov mapping: uses discrete log of (alpha^i + 1)
        log_table = {pow(alpha, i, p): i for i in range(p-1)}
        for i in range(p-1):
            val = (pow(alpha, i, p) + 1) % p
            if val == 0: s[i] = -1
            else:
                exponent = log_table[val]
                s[i] = 1 if (exponent % 2 == 0) else -1

        cand = np.roll(s[:n], n // 4)
        f = np.fft.fft(cand)
        acf = np.round(np.real(np.fft.ifft(f * np.conj(f)))).astype(int)
        curr_psl = int(np.max(np.abs(acf[1:n//2+1])))
        if curr_psl < best_psl:
            best_psl, best_seed, best_alpha = curr_psl, cand.astype(float), alpha

    return best_seed, p, best_alpha, n // 4

def run_pecan_lns():
    n = int(input("Enter N: "))

    # 1. Get Initial Sidelnikov
    seed, p, alpha, shift = get_best_sidelnikov_seed(n)
    init_f = np.fft.fft(seed)
    init_psl = int(np.max(np.abs(np.round(np.real(np.fft.ifft(init_f * np.conj(init_f))))[1:n//2+1])))

    print(f"Optimal Alpha Found: {alpha} | Initial PSL: {init_psl}")

    # 2. Optimization Logic
```

```
    curr = seed.copy()
    best_s = seed.copy()
    best_psl = init_psl
    stagnation = 0

    for i in range(1, 1001):
        S = np.fft.fft(curr)
        S_ideal = np.sqrt(n) * np.exp(1j * np.angle(S))
        s_cont = np.real(np.fft.ifft(S_ideal))
        cand = np.where(s_cont >= 0, 1.0, -1.0)

        f_cand = np.fft.fft(cand)
        c_psl = int(np.max(np.abs(np.round(np.real(np.fft.ifft(f_cand * np.conj(f_cand))))[1:n//2+1])))

        if c_psl < best_psl:
            best_psl, best_s, stagnation = c_psl, cand.copy(), 0
            print(f"  Iter {i} | PSL: {best_psl}")
        else:
            stagnation += 1

        # Local Neighborhood Search (Kick)
        if stagnation > 50:
            # Flip bits with lowest confidence (nearest to 0 in continuous domain)
            cand[np.abs(s_cont) < np.percentile(np.abs(s_cont), 2)] *= -1
            stagnation = 0
        curr = cand

    # 3. Report Generation
    filename = f"Sidelnikov_PeCAN_N{n}.txt"
    with open(filename, "w") as f:
        f.write(f"METHOD: PECAN + LNS HYBRID\n")
        f.write(f"N: {n} | Base Prime: {p} | Alpha (Primitive Root): {alpha}\n")
        f.write(f"Initial PSL: {init_psl} | Final PSL: {best_psl}\n")
```

```
            f.write("="*50 + "\n\n")

            f.write("--- INITIAL SEQUENCE (Sidelnikov Alpha {}) ---\n".format(alpha))
            s_str = "".join(['1' if x > 0 else '0' for x in seed])
            for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")

            f.write("\n--- FINAL OPTIMIZED SEQUENCE ---\n")
            b_str = "".join(['1' if x > 0 else '0' for x in best_s])
            for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")

        print(f"Report Saved to: {filename}")

    if __name__ == "__main__":
        run_pecan_lns()
```

HYBRID SIMULATED ANNEALING

A. LEGENDRE SEQUENCE

```
import numpy as np
import math
import os

def get_unified_seed(n):
    p = n
    while not all(p % i != 0 for i in range(2, int(p**0.5) + 1)): p += 1
    s = np.ones(p)
    for i in range(1, p):
        if pow(i, (p - 1) // 2, p) != 1: s[i] = -1
    return np.roll(s[:n], n // 4).astype(float), p

def get_psl(x):
```

```
    return
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(x)*np.conj(np.fft.fft(x)))))[1:])
))

def write_wrapped_bits(f, seq):
    bin_str = "".join(['1' if x > 0 else '0' for x in seq])
    for i in range(0, len(bin_str), 100):
        f.write(bin_str[i:i+100] + "\n")

def run_hybrid_sa():
    n = int(input("Enter N: ") or "1024")
    initial_seq, p = get_unified_seed(n)
    init_psl = get_psl(initial_seq)

    current, cur_psl = initial_seq.copy(), init_psl
    best_s, best_psl = initial_seq.copy(), init_psl
    temp = 45.0

    for i in range(15000):
        new = current.copy()
        idx = np.random.randint(0, n, size=np.random.randint(1, 4))
        new[idx] *= -1
        new_psl = get_psl(new)

        if new_psl < cur_psl or np.random.rand() < math.exp((cur_psl - new_psl)/temp):
            current, cur_psl = new, new_psl
            if cur_psl < best_psl:
                best_psl, best_s = cur_psl, current.copy()
                print(f"SA Iter {i} | PSL: {best_psl}")
        temp *= 0.9998

    filename = f"HybridSA_N{n}_PSL{best_psl}.txt"
    with open(filename, "w") as f:
        f.write(f"HYBRID SA REPORT | N: {n} | Base Prime: {p}\n")
```

```
            f.write(f"Initial PSL: {init_psl} | Final PSL: {best_psl}\n\n")
            f.write("--- INITIAL SEQUENCE ---\n")
            write_wrapped_bits(f, initial_seq)
            f.write("\n--- OPTIMIZED SEQUENCE ---\n")
            write_wrapped_bits(f, best_s)
        print(f"Saved: {filename}")

    if __name__ == "__main__": run_hybrid_sa()
```

B. WEIL SEQUENCE

```
    import numpy as np
    import math
    import os

    def get_best_weil_seed(n):
        # [Exactly same get_best_weil_seed logic as Method 1]
        def is_prime(num):
            if num < 2: return False
            for i in range(2, int(num**0.5) + 1):
                if num % i == 0: return False
            return True
        def get_primitive_roots(p):
            roots = []; phi = p - 1; temp = phi; factors = []; d = 2
            while d * d <= temp:
                if temp % d == 0:
                    factors.append(d)
                    while temp % d == 0: temp //= d
                d += 1
            if temp > 1: factors.append(temp)
            for res in range(2, p):
                ok = True
                for f in factors:
                    if pow(res, phi // f, p) == 1:
                        ok = False; break
```

```
            if ok: roots.append(res)
        return roots
    p = n
    while not is_prime(p): p += 1
    roots = get_primitive_roots(p)
    best_psl = float('inf'); best_seed = None; best_g = None
    for g in roots:
        s = np.ones(p)
        for i in range(p):
            val = (pow(i, g, p) + 1) % p
            s[i] = 1 if (val == 0 or pow(val, (p - 1) // 2, p) == 1) else -1
        cand = np.roll(s[:n], n // 4)
        curr_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(cand)*np.conj(np.fft.fft(cand))
)))[1:n//2+1])))
        if curr_psl < best_psl:
            best_psl, best_seed, best_g = curr_psl, cand.astype(float), g
    return best_seed, p, best_g, n // 4

def run_sa():
    n = int(input("Enter N: "))
    seed, p, g, shift = get_best_weil_seed(n)
    init_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(seed)*np.conj(np.fft.fft(seed))
)))[1:n//2+1])))

    curr = seed.copy(); cur_psl = init_psl; best_s = seed.copy(); best_psl = init_psl; temp
= 40.0
    for i in range(20000):
        new = curr.copy()
        if np.random.rand() < 0.8: new[np.random.randint(n)] *= -1
        else:
            sz = np.random.randint(2, 8); st = np.random.randint(0, n-sz)
            new[st:st+sz] *= -1
```

```
        n_psl = int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(new)*np.conj(np.fft.fft(new))))))[1:n//2+1]))
        if n_psl < cur_psl or np.random.rand() < math.exp((cur_psl - n_psl)/temp):
            curr, cur_psl = new, n_psl
            if cur_psl < best_psl:
                best_psl, best_s = cur_psl, curr.copy()
                print(f"Iter {i} | PSL: {best_psl}")
        temp *= 0.9998

    with open(f"SA_Weil_N{n}.txt", "w") as f:
        f.write(f"HYBRID SA | N={n} | Prime={p} | Root={g}\nInitial PSL: {init_psl} | Final: {best_psl}\n")
        s_str = "".join(['1' if x > 0 else '0' for x in seed])
        for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
        f.write("\n--- OPTIMIZED ---\n")
        b_str = "".join(['1' if x > 0 else '0' for x in best_s])
        for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
    print(f"Saved: SA_Weil_N{n}.txt")

if __name__ == "__main__": run_sa()
```

C. SIDELNIKOV SEQUENCE

```
import numpy as np
import math
import os

def get_best_sidelnikov_seed(n):
    def is_prime(num):
        if num < 2: return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0: return False
        return True
```

```
def get_primitive_roots(p):
    roots = []; phi = p - 1; temp = phi; factors = []; d = 2
    while d * d <= temp:
        if temp % d == 0:
            factors.append(d)
            while temp % d == 0: temp //= d
        d += 1
    if temp > 1: factors.append(temp)
    for res in range(2, p):
        ok = True
        for f in factors:
            if pow(res, phi // f, p) == 1:
                ok = False; break
        if ok: roots.append(res)
    return roots

p = n
while not is_prime(p): p += 1
all_roots = get_primitive_roots(p)
roots_to_scan = all_roots[:25]

best_psl = float('inf'); best_seed = None; best_alpha = None

for alpha in roots_to_scan:
    s = np.ones(p)
    log_table = {pow(alpha, i, p): i for i in range(p-1)}
    for i in range(p-1):
        val = (pow(alpha, i, p) + 1) % p
        if val == 0: s[i] = -1
        else:
            exponent = log_table[val]
            s[i] = 1 if (exponent % 2 == 0) else -1
```

```
            cand = np.roll(s[:n], n // 4)
            f = np.fft.fft(cand)
            acf = np.round(np.real(np.fft.ifft(f * np.conj(f)))).astype(int)
            curr_psl = int(np.max(np.abs(acf[1:n//2+1])))
            if curr_psl < best_psl:
                best_psl, best_seed, best_alpha = curr_psl, cand.astype(float), alpha

    return best_seed, p, best_alpha, n // 4

def run_hybrid_sa():
    n = int(input("Enter N: "))
    seed, p, alpha, shift = get_best_sidelnikov_seed(n)

    def get_psl(x):
        f = np.fft.fft(x)
        acf = np.round(np.real(np.fft.ifft(f * np.conj(f)))).astype(int)
        return int(np.max(np.abs(acf[1:n//2+1])))

    init_psl = get_psl(seed)
    curr = seed.copy(); cur_psl = init_psl
    best_s = seed.copy(); best_psl = init_psl
    temp = 35.0

    print(f"Optimal Alpha: {alpha} | Initial PSL: {init_psl}")

    for i in range(20000):
        new = curr.copy()
        # Single bit flip
        new[np.random.randint(n)] *= -1

        n_psl = get_psl(new)
        delta = n_psl - cur_psl

        # Metropolis Criterion
```

```
            if delta < 0 or np.random.rand() < math.exp(-delta/temp):
                curr, cur_psl = new, n_psl
                if cur_psl < best_psl:
                    best_psl, best_s = cur_psl, curr.copy()
                    print(f"  SA Iter {i} | PSL: {best_psl}")
            temp *= 0.9997

        filename = f"Sidelnikov_SA_N{n}.txt"
        with open(filename, "w") as f:
            f.write(f"METHOD: HYBRID SIMULATED ANNEALING\n")
            f.write(f"N: {n} | Base Prime: {p} | Alpha: {alpha}\n")
            f.write(f"Initial PSL: {init_psl} | Final PSL: {best_psl}\n\n")
            f.write("--- INITIAL SEQUENCE ---\n")
            s_str = "".join(['1' if x > 0 else '0' for x in seed])
            for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
            f.write("\n--- FINAL OPTIMIZED SEQUENCE ---\n")
            b_str = "".join(['1' if x > 0 else '0' for x in best_s])
            for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
        print(f"Report Saved: {filename}")

    if __name__ == "__main__":
        run_hybrid_sa()
```

GENETIC ALGORITHM

A. LEGENDRE SEQUENCE

```
import numpy as np
import os

def get_unified_seed(n):
    p = n
    while not all(p % i != 0 for i in range(2, int(p**0.5) + 1)): p += 1
    s = np.ones(p)
    for i in range(1, p):
```

```
        if pow(i, (p - 1) // 2, p) != 1: s[i] = -1
    return np.roll(s[:n], n // 4).astype(float), p

def get_psl(x):
    return
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(x)*np.conj(np.fft.fft(x)))))[1:])
))

def write_wrapped_bits(f, seq):
    bin_str = "".join(['1' if x > 0 else '0' for x in seq])
    for i in range(0, len(bin_str), 100):
        f.write(bin_str[i:i+100] + "\n")

def run_ga():
    n = int(input("Enter N: ") or "1024")
    initial_seed, p = get_unified_seed(n)
    init_psl = get_psl(initial_seed)

    pop_size = 10
    population = [np.roll(initial_seed, i * (n//pop_size)) for i in range(pop_size)]

    for gen in range(60):
        population.sort(key=get_psl)
        best_in_pop = population[0]
        print(f"Gen {gen} | Top PSL: {get_psl(best_in_pop)}")

        next_gen = population[:2]
        while len(next_gen) < pop_size:
            p1, p2 = population[0], population[np.random.randint(1, 4)]
            cp = np.random.randint(n)
            child = np.concatenate([p1[:cp], p2[cp:]])
            if np.random.rand() < 0.15: child[np.random.randint(n)] *= -1
            next_gen.append(child)
        population = next_gen
```

```
    final_seq = population[0]
    final_psl = get_psl(final_seq)
    filename = f"GA_N{n}_PSL{final_psl}.txt"
    with open(filename, "w") as f:
        f.write(f"GA REPORT | N: {n} | Base Prime: {p}\n")
        f.write(f"Initial Seed PSL: {init_psl} | Final PSL: {final_psl}\n\n")
        f.write("--- INITIAL SEED SEQUENCE ---\n")
        write_wrapped_bits(f, initial_seed)
        f.write("\n--- OPTIMIZED SEQUENCE ---\n")
        write_wrapped_bits(f, final_seq)
    print(f"Saved: {filename}")

if __name__ == "__main__": run_ga()
```

B. WEIL SEQUENCE

```
import numpy as np
import os

def get_best_weil_seed(n):
    # [Exactly same get_best_weil_seed logic as Method 1]
    def is_prime(num):
        if num < 2: return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0: return False
        return True
    def get_primitive_roots(p):
        roots = []; phi = p - 1; temp = phi; factors = []; d = 2
        while d * d <= temp:
            if temp % d == 0:
                factors.append(d)
                while temp % d == 0: temp //= d
            d += 1
        if temp > 1: factors.append(temp)
        for res in range(2, p):
```

```
            ok = True
            for f in factors:
                if pow(res, phi // f, p) == 1:
                    ok = False; break
            if ok: roots.append(res)
        return roots
    p = n
    while not is_prime(p): p += 1
    roots = get_primitive_roots(p)
    best_psl = float('inf'); best_seed = None; best_g = None
    for g in roots:
        s = np.ones(p)
        for i in range(p):
            val = (pow(i, g, p) + 1) % p
            s[i] = 1 if (val == 0 or pow(val, (p - 1) // 2, p) == 1) else -1
        cand = np.roll(s[:n], n // 4)
        curr_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(cand)*np.conj(np.fft.fft(cand))
)))[1:n//2+1])))
        if curr_psl < best_psl:
            best_psl, best_seed, best_g = curr_psl, cand.astype(float), g
    return best_seed, p, best_g, n // 4

def run_ga():
    n = int(input("Enter N: "))
    seed, p, g, shift = get_best_weil_seed(n)
    init_psl =
int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(seed)*np.conj(np.fft.fft(seed))
)))[1:n//2+1])))

    pop = [np.roll(seed, i*5) for i in range(12)]
    for gen in range(60):
```

```
        pop.sort(key=lambda x: int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(x)*np.conj(np.fft.fft(x)))))[1:n//2+1]))))
        print(f"Gen {gen} | Top PSL: {int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(pop[0])*np.conj(np.fft.fft(pop[0])))))[1:n//2+1])))}")
        next_gen = pop[:2]
        while len(next_gen) < 12:
            p1, p2 = pop[0], pop[np.random.randint(1, 4)]
            cp = np.random.randint(n)
            child = np.concatenate([p1[:cp], p2[cp:]])
            if np.random.rand() < 0.2: child[np.random.randint(n)] *= -1
            next_gen.append(child)
        pop = next_gen

    final_psl = int(np.max(np.abs(np.round(np.real(np.fft.ifft(np.fft.fft(pop[0])*np.conj(np.fft.fft(pop[0])))))[1:n//2+1])))
    with open(f"GA_Weil_N{n}.txt", "w") as f:
        f.write(f"GA | N={n} | Prime={p} | Root={g}\nInitial PSL: {init_psl} | Final: {final_psl}\n")
        s_str = "".join(['1' if x > 0 else '0' for x in seed])
        for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
        f.write("\n--- OPTIMIZED ---\n")
        b_str = "".join(['1' if x > 0 else '0' for x in pop[0]])
        for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
    print(f"Saved: GA_Weil_N{n}.txt")

if __name__ == "__main__": run_ga()
```

C. SIDELNIKOV SEQUENCE

```
import numpy as np
import os
```

```
def get_best_sidelnikov_seed(n):
    def is_prime(num):
        if num < 2: return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0: return False
        return True

    def get_primitive_roots(p):
        roots = []; phi = p - 1; temp = phi; factors = []; d = 2
        while d * d <= temp:
            if temp % d == 0:
                factors.append(d)
                while temp % d == 0: temp //= d
            d += 1
        if temp > 1: factors.append(temp)
        for res in range(2, p):
            ok = True
            for f in factors:
                if pow(res, phi // f, p) == 1:
                    ok = False; break
            if ok: roots.append(res)
        return roots

    p = n
    while not is_prime(p): p += 1
    all_roots = get_primitive_roots(p)
    roots_to_scan = all_roots[:25]

    best_psl = float('inf'); best_seed = None; best_alpha = None

    for alpha in roots_to_scan:
        s = np.ones(p)
        log_table = {pow(alpha, i, p): i for i in range(p-1)}
        for i in range(p-1):
```

```
            val = (pow(alpha, i, p) + 1) % p
            if val == 0: s[i] = -1
            else:
                exponent = log_table[val]
                s[i] = 1 if (exponent % 2 == 0) else -1

        cand = np.roll(s[:n], n // 4)
        f = np.fft.fft(cand)
        acf = np.round(np.real(np.fft.ifft(f * np.conj(f)))).astype(int)
        curr_psl = int(np.max(np.abs(acf[1:n//2+1])))
        if curr_psl < best_psl:
            best_psl, best_seed, best_alpha = curr_psl, cand.astype(float), alpha

    return best_seed, p, best_alpha, n // 4

def run_ga():
    n = int(input("Enter N: "))
    seed, p, alpha, shift = get_best_sidelnikov_seed(n)

    def get_psl(x):
        f = np.fft.fft(x)
        acf = np.round(np.real(np.fft.ifft(f * np.conj(f)))).astype(int)
        return int(np.max(np.abs(acf[1:n//2+1])))

    init_psl = get_psl(seed)
    print(f"Optimal Alpha: {alpha} | Initial PSL: {init_psl}")

    # Initialization: Variety of rotations of the best seed
    pop_size = 12
    population = [np.roll(seed, i*3) for i in range(pop_size)]

    for gen in range(60):
        # Sort by PSL
        population.sort(key=get_psl)
```

```
        top_psl = get_psl(population[0])
        print(f"  Gen {gen} | Best PSL: {top_psl}")

        next_gen = population[:2] # Elitism (keep top 2)

        while len(next_gen) < pop_size:
            # Crossover
            p1, p2 = population[0], population[np.random.randint(1, 5)]
            cp = np.random.randint(n)
            child = np.concatenate([p1[:cp], p2[cp:]])

            # Mutation
            if np.random.rand() < 0.15:
                child[np.random.randint(n)] *= -1
            next_gen.append(child)
        population = next_gen

    final_seq = population[0]
    final_psl = get_psl(final_seq)

    filename = f"Sidelnikov_GA_N{n}.txt"
    with open(filename, "w") as f:
        f.write(f"METHOD: GENETIC ALGORITHM\n")
        f.write(f"N: {n} | Base Prime: {p} | Alpha: {alpha}\n")
        f.write(f"Initial PSL: {init_psl} | Final PSL: {final_psl}\n\n")
        f.write("--- INITIAL SEQUENCE ---\n")
        s_str = "".join(['1' if x > 0 else '0' for x in seed])
        for i in range(0, len(s_str), 100): f.write(s_str[i:i+100] + "\n")
        f.write("\n--- FINAL OPTIMIZED SEQUENCE ---\n")
        b_str = "".join(['1' if x > 0 else '0' for x in final_seq])
        for i in range(0, len(b_str), 100): f.write(b_str[i:i+100] + "\n")
    print(f"Report Saved: {filename}")
if __name__ == "__main__":
    run_ga()
```

REFERENCES

[1] S. W. Golomb and R. A. Schholtz, "Generalized Barker sequences," *IEEE Trans. Inf. Theory*, vol. 11, no. 4, pp. 533–537, Oct. 1965.

[2] T. Helleseth and K. Yang, "On the correlation properties of quadratic residue sequences," *IEEE Trans. Inf. Theory*, vol. 47, no. 7, pp. 2879–2885, Nov. 2001.

[3] L. R. Welch, "Lower bounds on the maximum cross correlation of signals," *IEEE Trans. Inf. Theory*, vol. 20, no. 3, pp. 397–399, May 1974.

[4] D. Sarwate and M. Pursley, "Crosscorrelation properties of pseudorandom and related sequences," *Proc. IEEE*, vol. 68, no. 5, pp. 593–619, May 1980.

[5] T. Helleseth, "Some results about the correlation properties of linear feedback shift register sequences," *Proc. IEEE*, vol. 73, no. 4, pp. 658–663, Apr. 1985.

[6] J. H. Holland, "Adaptation in natural and artificial systems," *University of Michigan Press*, 1975.

[7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[8] K. A. De Jong, "Adaptive evolutionary computation," *IEE Proc. Software*, vol. 144, no. 3, pp. 148–153, 1997.

[9] J. Brest, S. Greiner, B. Bošković, and V. Žumer, "Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark functions," *IEEE Trans. Evol. Comput.*, vol. 10, no. 6, pp. 646–657, Dec. 2006.

[10] D. E. Goldberg and J. H. Holland, "Genetic algorithms and machine learning," *Machine Learning*, vol. 3, no. 2, pp. 95–99, 1988.